Einführung in die Programmierung

Prof. Dr. Peter Thiemann
Marius Weidner
Simon Dorer, Moritz Steffl

Universität Freiburg
Institut für Informatik
Wintersemester 2026

Übungsblatt 4

Abgabe: Dienstag, 17.11.2026, 10:00 Uhr

Aufgabe 4.1 (Sequenzen; 3 Punkte; Dateien: `sequences.py`, `test_sequences.py`)

Entwickeln Sie die folgenden Funktionen nach der Konstruktionsanleitung aus der Vorlesung. Schreiben Sie dabei für jede Funktion zuerst die Tests und erst danach die Implementierung.

Gehen Sie für jede Funktion wie folgt vor:

1. Schreiben Sie eine kurze Beschreibung (d. h. einen *kurzen* Kommentar) und den Funktionskopf mit Typannotationen in die Datei `sequences.py`. Verwenden Sie zunächst `...` als Platzhalter für den Funktionsrumpf.
2. Überlegen Sie sich geeignete Testfälle, die sowohl gewöhnliche Fälle als auch Randfälle abdecken. Bestimmen Sie die erwarteten Ergebnisse von Hand und formulieren Sie die Tests mit `assert`-Anweisungen in der Datei `test_sequences.py`. Importieren Sie dazu die zu testende Funktion aus `sequences.py`. Benennen Sie die Testfunktion nach dem Schema `test_funktionsname`, also beispielsweise `test_filter_range`.
3. Entwickeln Sie anschließend das Gerüst und den vollständigen Funktionsrumpf, indem Sie den Platzhalter `...` durch die tatsächliche Implementierung ersetzen. Führen Sie Ihre Tests aus und korrigieren Sie gegebenenfalls Ihre Implementierung.

Pytest installieren: Installieren Sie `pytest` in einer *virtuellen Umgebung* (`venv`)[1]. Unter Ubuntu (auch in WSL) installieren Sie die Unterstützung dafür einmalig mit `sudo apt-get install python3.14-venv`. Unter macOS ist `venv` in der zuvor installierten Python-3.14-Version bereits enthalten.

Führen Sie dann im VSCode-Terminal *im Ordner dieses Übungsblatts* folgende Befehle aus:

```
python3.14 -m venv .venv
source .venv/bin/activate
python -m pip install pytest
```

Wählen Sie anschließend die virtuelle Umgebung als Python-Interpreter für VSCode aus. Öffnen Sie hierzu mit der Taste `F1` die Befehlspalette und wählen Sie den Befehl *Python: Select Interpreter*. Wählen Sie dort den Interpreter `.venv/bin/python` aus, damit VSCode die Tests in dieser Umgebung ausführt.

[1] https://docs.python.org/3.14/library/venv.html

Erstellen Sie die virtuelle Umgebung `.venv` nur einmal.

Aktivieren Sie die Umgebung in jedem neuen Terminal erneut mit dem zweiten Befehl (`source .venv/bin/activate`). Wechseln Sie dazu zunächst in den Ordner dieses Übungsblatts. Die Installation von `pytest` müssen Sie nicht wiederholen.

Mit dem Befehl `deactivate` können Sie die virtuelle Umgebung im aktuellen Terminal wieder deaktivieren.

Geben Sie Ihre virtuelle Umgebung `.venv` nicht mit ab. Die Webplattform verhindert dies bereits automatisch, falls Sie `git` über das Terminal verwenden.

Tests ausführen: Speichern Sie Ihre Dateien und öffnen Sie in VSCode links über das Test-Symbol den Test-Explorer und wählen Sie *Alle Tests ausführen* (▷). Wählen Sie bei der ersten Einrichtung *Python-Tests konfigurieren*, dann *pytest* und den Ordner dieses Übungsblatts.

(a) (1.5 Punkte) Schreiben Sie eine Funktion `filter_range`, die eine Liste von ganzen Zahlen `xs` und zwei ganze Zahlen `lower` und `upper` als Argumente nimmt. Die Funktion soll eine neue Liste zurückgeben, die alle Zahlen `x` aus `xs` enthält, für die `x` im Intervall [`lower`, `upper`] liegt. Die beiden Intervallgrenzen `lower` und `upper` sind dabei im Intervall enthalten.

(b) (1.5 Punkte) Schreiben Sie eine Funktion `dot_product`, die zwei gleich lange Tupel von ganzen Zahlen `xs` und `ys` als Argumente nimmt und das Skalarprodukt der durch diese Tupel dargestellten Vektoren berechnet und zurückgibt. Das Skalarprodukt zweier Vektoren $xs = \langle x_0, x_1, \ldots, x_{n-1} \rangle$ und $ys = \langle y_0, y_1, \ldots, y_{n-1} \rangle$ lässt sich wie folgt berechnen:

$$\sum_{i=0}^{n-1} x_i \cdot y_i$$

Das Skalarprodukt zweier nulldimensionaler Vektoren (Länge 0) ist als 0 definiert.

Aufgabe 4.2 (Mathematische Konstante π; 3 Punkte; Datei: `pi.py`)
Die mathematische Konstante π kann durch die Leibniz-Reihe

$$\pi = 4 \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1}$$

dargestellt werden.

In der Praxis lässt sich diese unendliche Summe nicht exakt berechnen. Wir können jedoch eine Annäherung $S_m \approx \pi$ berechnen, indem wir die ersten $m+1$ Summanden der Reihe aufsummieren und mit 4 multiplizieren:

$$S_m = 4 \sum_{n=0}^{m} \frac{(-1)^n}{2n+1}$$

Schreiben Sie eine Funktion `approx_pi`, die eine ganze Zahl `m` als Argument nimmt und die m-te Annäherung S_m berechnet und zurückgibt. Sie dürfen davon ausgehen, dass `m` nicht negativ ist.

Zum Beispiel:

```
>>> approx_pi(0)
4.0
>>> approx_pi(1)
2.666666666666667
>>> approx_pi(42)
3.1648453252882898
>>> approx_pi(1337)
3.1408452695387057
>>> approx_pi(1_000_000)
3.1415936535887745
>>> math.isclose(approx_pi(320_000_000), math.pi)  # Takes some time :)
True
```

Aufgabe 4.3 (Vigenère-Chiffre; 4 Punkte; Datei: `crypto.py`)

Eine einfache Möglichkeit, Texte zu verschlüsseln, ist die Vigenère-Chiffre[2]. Dabei wird jeder Buchstabe des Textes um eine bestimmte Anzahl von Stellen im Alphabet verschoben. Die Verschiebungen werden durch die Buchstaben eines Schlüssel-Wortes festgelegt. In dieser Aufgabe sollen Sie diese Verschlüsselung in zwei Schritten implementieren. Gehen Sie dabei wie folgt vor:

(a) (1.5 Punkte) Schreiben Sie eine Funktion `encode_char`, die zwei *Buchstaben* (Typ: `str`) `char` und `key_char` als Argumente nimmt und den mit dem Schlüsselbuchstaben `key_char` verschlüsselten Buchstaben `char` zurückgibt.

Die Verschlüsselung funktioniert dabei wie folgt: Wir verschieben `char` im Alphabet in Richtung `Z`. Die Anzahl der Stellen entspricht der Position von `key_char` im Alphabet, wobei ab `0` gezählt wird: `A` bedeutet keine Verschiebung, `B` eine Stelle und `Z` 25 Stellen. Nach `Z` wird wieder bei `A` begonnen.

Sie dürfen davon ausgehen, dass `char` und `key_char` aus genau einem Großbuchstaben von `A` bis `Z` bestehen.

Beispielsweise erhalten wir somit für `char = "T"` und `key_char = "M"` den verschlüsselten Buchstaben `"F"`:

[2] Wikipedia: Vigenère-Chiffre

$$\underbrace{\texttt{T}}_{19} + \underbrace{\texttt{M}}_{12} = \underbrace{\texttt{F}}_{31 \bmod 26 = 5} .$$

Hinweis: Erinnern Sie sich an die Funktionen `ord` und `chr` aus der Vorlesung.

Zum Beispiel:

```
>>> encode_char("A", "A")
'A'
>>> encode_char("A", "L")
'L'
>>> encode_char("T", "M")
'F'
>>> encode_char("Z", "B")
'A'
>>> encode_char("Z", "Z")
'Y'
```

(b) (2.5 Punkte) Schreiben Sie eine Funktion `encode_message`, die zwei Strings `message` und `key` als Argumente nimmt und den mit der Vigenère-Chiffre verschlüsselten Text zurückgibt.

Jeder Buchstabe von `message` wird dabei mit dem Buchstaben aus `key` an derselben Position verschlüsselt. Ist das Ende von `key` erreicht, wird der Schlüssel wieder von vorne verwendet. Ist der Text kürzer als der Schlüssel, werden die übrigen Schlüsselbuchstaben nicht verwendet.

Verwenden Sie in Ihrer Implementierung die Funktion `encode_char` aus Aufgabenteil (a).

Sie dürfen davon ausgehen, dass sowohl `message` als auch `key` nur Großbuchstaben von `A` bis `Z` enthalten und `key` nicht leer ist.

Zum Beispiel wird der Text `"ATTACKATDAWN"` mit dem Schlüssel `"LEMON"` wie folgt verschlüsselt:

Text:	A	T	T	A	C	K	A	T	D	A	W	N
Schlüssel:	L	E	M	O	N	L	E	M	O	N	L	E
Ergebnis:	L	X	F	O	P	V	E	F	R	N	H	R

Einige Beispiele:

```
>>> encode_message("", "LEMON")
''
>>> encode_message("ATTACKATDAWN", "LEMON")
'LXFOPVEFRNHR'
>>> encode_message("HELLOWORLD", "ABC")
'HFNLPYOSND'
>>> encode_message("ABC", "A")
'ABC'
```

```
>>> encode_message("AB", "XYZ")
'XZ'
```

Aufgabe 4.4 (Erfahrungen; Datei: `NOTES.md`)

Notieren Sie Ihre Erfahrungen mit diesem Übungsblatt (benötigter Zeitaufwand, Probleme, Bezug zur Vorlesung, Interessantes, etc.).

Editieren Sie hierzu die Datei `NOTES.md` im Abgabeordner dieses Übungsblattes auf unserer Webplattform. Halten Sie sich an das dort vorgegebene Format, da wir den Zeitbedarf mit einem Python-Skript automatisch statistisch auswerten. Die Zeitangabe `5.5 h` steht für 5 Stunden 30 Minuten.

Der Build-Server überprüft ebenfalls, ob Sie das Format korrekt angegeben haben. Prüfen Sie, ob der Build-Server mit Ihrer Abgabe zufrieden ist, so wie es im Skript zum Abgabesystem gezeigt wurde.